

מדריך ההתקנה של הקורס – macOS, Windows

זמן משוער: 45–60 דקות

מטרה

לסיים את השיעור הראשון עם סביבת עבודה שעובדת מקצה לקצה: VS Code עם התוספים, Git, Node, Live Server, וחשבון גיטהאב עם מאגר פרטי משלך שכבר קיבל דחיפה ראשונה מוצלחת.

עבוד לפי המסלול של מערכת ההפעלה שלך בלבד. כל פרק כאן מחולק לשני מסלולים – **macOS** ו-**Windows** – והם לא מתערבבים. אם אתה על מק, דלג על כל מה שמסומן Windows, ולהפך.

עשה את זה עכשיו, בבית ובכיתה, כשאתה לידך. בשבוע 7 וב-8 העומס אחר, ותקלת התקנה שם עולה לך שבוע.

אם משהו נשבר

נספח א בסוף המדריך מכיל 14 הודעות שגיאה – עם הטקסט המדויק שתראה על המסך, כדי שתוכל לחפש אותו. אל תנחש: מצא את ההודעה שלך שם.

0. מה יהיה לך בסוף

- ☐ VS Code מותקן, ו-8 התוספים המומלצים מותקנים בו
 - ☐ Live Server רץ, והעמוד נפתח בכתובת `http://127.0.0.1:5500`
 - ☐ `node -v` ו-`npm -v` עונים
 - ☐ `git --version` עונה, והשם והמייל שלך מוגדרים
 - ☐ אתה יודע לפתוח את כלי המפתחים (DevTools) ב-Chrome, ומוצא Console ו-Network Elements
 - ☐ חשבון גיטהאב חינומי, ומאגר פרטי בשם `frontend-2027-שליך`
 - ☐ אני מופיע ב-Collaborators של המאגר הזה (Pending invite זה תקין)
 - ☐ דחיפה ראשונה עברה בהצלחה
 - ☐ מאגר החומרים הציבורי משוכפל למחשב שלך
 - ☐ Claude Code מותקן – רק למי שיש מנוי בתשלום (לא נחוץ עד שבוע 8; בלי מנוי, הצ'אט החינומי ומאותו ציון)
 - ☐ צילום המסך של רשימת הבדיקה (נספח ב) – הוא שווה נקודות במטלה הראשונה
- חשבון גיטהאב חינומי מספיק לכל הקורס.** אין צורך בארגון, אין צורך באימות סטודנט, ואין שום שלב בקורס שדורש מנוי בתשלום.

1. התקנת VS Code

זה העורך היחיד שנשתמש בו בקורס. לא כי אחרים גרועים — אלא כי כל ההגדרות, התוספים והמדריכים כאן בנויים עליו, ואין מתרגל שיתאים אותם לעורך אחר.

מסלול macOS

1. גש אל <https://code.visualstudio.com> ולחץ על כפתור ההורדה. האתר מזהה לבד שאתה על מק.
2. פתח את קובץ ה-ZIP שירד. תקבל אפליקציה בשם Visual Studio Code.
3. גרור אותה לתיקיית Applications. אם תפעיל אותה מתיקיית ההורדות, חלק מהדברים יעבדו מוזר בהמשך.
4. הפעל אותה. אם מוצגת אזהרה שהאפליקציה הורדה מהאינטרנט — אשר.
5. חשוב: פתח את לוח הפקודות ב- `Cmd+Shift+P`, הקלד `shell command`, ובחר `Shell Command: Install 'code' command in PATH`. בלי זה לא תוכל לפתוח תיקייה מהטרמינל, וזה יחזור אליך בפרק 6.

מסלול Windows

1. גש אל <https://code.visualstudio.com> ולחץ על כפתור ההורדה. הורד את גרסת ה-**User Installer** (bit-64) — היא לא דורשת הרשאות מנהל.
2. הפעל את קובץ ההתקנה.
3. במסך `Select Additional Tasks` סמן את שתי התיבות האלה:
 - `Add "Open with Code" action to Windows Explorer file context menu`
 - `Add to PATH (requires shell restart)` השנייה היא החשובה. בלעדיה תקבל בהמשך `'code' is not recognized`.
4. סיים את ההתקנה והפעל את VS Code.

2. התוספים — ומה כל אחד עושה

אין צריך לחפש אותם. בכל תיקייה שתקבל בקורס יש קובץ `vscode/extensions.json`, ובפעם הראשונה שתפתח את התיקייה VS Code יציג בפינה הודעה: "This workspace has extension recommendations". לחץ **Install All**. אם ההודעה לא הופיעה: פתח את לוח התוספים (`Cmd+Shift+X` במק, `Ctrl+Shift+X` בחלונות), הקלד בשורת החיפוש `@recommended`, ולחץ על כפתור ההתקנה ליד הקבוצה.

התוסף	בשביל מה הוא
Live Server	מריץ את העמוד דרך שרת אמיתי. זה הכלל של הקורס, ופרק 3 מסביר למה
Prettier	מסדר את הקוד בכל שמירה. אתה לא מפסיד נקודות על הזחה, וההבדלים בקומיטים נשארים
ESLint	תופס את טעויות ה-JavaScript האמיתיות — משתנה שלא בשימוש, <code>==</code> במקום <code>===</code> — לפני הדפדפן
Tailwind CSS IntelliSense	משבוע 6: השלמה אוטומטית ותצוגה מקדימה למחלקות של Tailwind
Error Lens	מציג את השגיאה על השורה עצמה, לא מוחבאת בפאנל. בקורס בלי מתרגל, זה התוסף הכי שימושי כאן

התוסף	בשביל מה הוא
HTMLHint	ולידציה של HTML תוך כדי הקלדה – אותן בדיקות שנריץ בשבוע 1
GitLens	מראה מי שינה כל שורה ומתי. הופך את "מה שיניתי ומתי" למשהו שרואים
Path Intellisense	משלים נתיבים יחסיים. הבאג הכי נפוץ בשבוע 1 הוא <code>src</code> או <code>href</code> שבור

התוספים האלה הם לא קישוט. **HTMLHint-ו Error Lens יחסכו לך את רוב הזמן שהיית מבזבז על שגיאות כתיב בשבועות הראשונים.**

3. Live Server – והכלל שלא משתנה

הכלל הכי חשוב מהשבוע הראשון

עמוד נפתח תמיד דרך **Live Server**. **אף פעם לא בלחיצה כפולה על הקובץ**. לחיצה כפולה פותחת אותו בכתובת `file://`, וזה שובר את `fetch` (שבוע 11), מודולים של ES (שבוע 8) ו-`localStorage` (שבוע 11). ההרגל שנוצר בשבוע 1 מתפוצץ בשבוע 11.

איך מריצים (זהה בשני המסלולים)

1. פתח ב-VS Code את התיקיה, לא קובץ בודד: `File > Open Folder...`
 2. לחץ לחיצה ימנית על `index.html` בסייר הקבצים.
 3. בחר **Open with Live Server**.
 4. הדפדפן ייפתח בכתובת `http://127.0.0.1:5500/index.html`.
- הסתכל בשורת הכתובת.** אם כתוב שם `file:///` – סגור את הלשונית ונסה שוב. אם כתוב `http://127.0.0.1:5500` – אתה בסדר. בשורת הסטטוס הכחולה בתחתית VS Code יופיע גם הכיתוב `Port : 5500`; לחיצה עליו עוצרת את השרת.
- הפורט 5500 נעול בהגדרות שקיבלת** (`.vscode/settings.json`). אל תשנה אותו. בשבוע 11 נשמור נתונים בדפדפן, והם שייכים לכתובת המדויקת שבה העמוד רץ – אם הפורט זז, הנתונים "נעלמים".
- עדיין לא צריך להבין את המילים `fetch`, `origin` או `localStorage`. תיפגש בהן שוב בשבוע 11, וההרגל כבר יהיה נכון.

4. Node.js – גרסת LTS

Node לא נחוץ לכתיבת HTML, אבל הבדיקות האוטומטיות רצות עליו – גם אצלך במחשב וגם במאגר שלך בכל דחיפה. **התקן את גרסת ה-LTS**, לא את גרסת ה-Current.

מסלול macOS

1. גש אל <https://nodejs.org> והורד את החבילה המסומנת **LTS** (קובץ `.pkg`).
2. הפעל את הקובץ ואשר את כל השלבים.
3. **פתח טרמינל חדש** – לא כזה שהיה פתוח מקודם – והרץ:

```
node -v
npm -v
```

אתה אמור לראות שתי שורות, כל אחת מתחילה במספר. למשל `v22.11.0` ו-`10.9.0`.

אם קיבלת `zsh: command not found: node`: הטרמינל לא יודע איפה Node הותקן. סגור את כל חלונות הטרמינל ופתח חדש. אם זה לא עזר, ראה תקלה **א-1** בנספח.

מסלול Windows

1. גש אל <https://nodejs.org> והורד את החבילה המסומנת **LTS** (קובץ `.msi`).
2. הפעל את קובץ ההתקנה ואשר את כל השלבים. **השאר את התיבה Add to PATH מסומנת** — היא מסומנת כברירת מחדל.
3. במסך **Tools for Native Modules** אין צורך לסמן כלום. אנחנו לא מקמפלים כלום בקורס הזה.
4. **סגור לגמרי את VS Code ופתח אותו מחדש**. זה לא טקס — משתני סביבה נטענים כשהתוכנה עולה, וטרמינל שהיה פתוח לפני ההתקנה לא יראה את Node לעולם.
5. פתח טרמינל משולב (`Ctrl+``) והרץ:

```
node -v
npm -v
```

אם קיבלת

`'node' is not recognized as an internal or external command, operable program or batch file.`
ראה תקלה **א-2** בנספח.

Git 5

מסלול macOS

Git כבר קיים על רוב המקינטשים. בדוק:

```
git --version
```

אם קיבלת מספר גרסה — אתה מסודר. אם נפתח חלון שמציע להתקין **Command Line Developer Tools**, אשר והמתן; זה לוקח כמה דקות.

מסלול Windows

1. גש אל <https://git-scm.com/download/win> והורדה תתחיל לבד.
2. הפעל את קובץ ההתקנה. **רוב המסכים אפשר לאשר כמו שהם**, חוץ משניים:
 - **Choosing the default editor used by Git** — בחר **Use Visual Studio Code as Git's default editor**. ברירת המחדל היא Vim, ומי שנקלע לתוכו בטעות מתקשה לצאת.
 - **Adjusting the name of the initial branch** — בחר **Override the default branch name** והשאר `main`.

```
git --version
```

הגדרות שחייבים לעשות פעם אחת (שני המסלולים)

Git חותם כל קומיט בשם ובמייל שלך. בלי זה הוא פשוט יסרב לעשות קומיט.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

השתמש **באותו מייל** שאיתו תפתח את חשבון הגיטהב, אחרת הקומיטים שלך לא יקושרו לחשבון שלך.

שורות ומעברי שורה – ההגדרה שאף אחד לא מסביר

:macOS

```
git config --global core.autocrlf input
```

:Windows

```
git config --global core.autocrlf true
```

בדוק שהכול נקלט:

```
git config --global --list
```

ומה **gitattributes**. עושה בשבילך

בכל תיקייה שתקבל בקורס יש קובץ בשם **gitattributes**, ובו שורה אחת:

```
* text=auto eol=lf
```

הרקע: מק ו-Windows מסמנים סוף שורה אחרת. בלי השורה הזו, קובץ שנערך במק ואז בחלונות נראה ל-Git כאילו **כל שורה בו**

השתנתה – גם אם שינית תו אחד. זה הורס כל אפשרות לקרוא מה השתנה, ושובר את הבדיקות האוטומטיות שמשוות קבצים.

השורה הזו אומרת ל-Git: אחסן הכול עם סוף שורה אחיד. **אתה לא צריך לעשות איתה כלום** – היא כבר בתיקייה. רק אל תמחק אותה,

ואל תתרגש מהאזהרה **warning: LF will be replaced by CRLF** בחלונות; היא תקינה (תקלה א-10).

5.A. פתיחת כלי המפתחים (DevTools)

כלי המפתחים הם הפאנל שבנוי בתוך Chrome ומראה מה הדפדפן עשה עם הקוד שכתבת. **בשיעור הראשון פותחים אותו כבר בדקה**

הראשונה של ההקלדה – עשה את זה עכשיו על כל עמוד, כדי שלא תחפש אותו אז.

פתיחה (שלוש דרכים, אותו דבר): **macOS** **Cmd+Opt+I** · **Windows** **F12** או **Ctrl+Shift+I** · בשתיקה: קליק ימני על העמוד,

ואז **בדוק** (Inspect). אותו קיצור סוגר אותו. **עגינה:** בפינה העליונה של הפאנל, שלוש נקודות אנכיות, ואז **Dock side** – ימין, שמאל,

למטה, או חלון נפרד. למטה נוח למסך רחב, ימין למסך צר. **קונסול נוסף בלי לעזוב לשונית:** Esc פותח ומסתיר את הקונסול בתחתית כל לשונית. **לשוניות שלא רואים:** הפאנל צר? לשוניות נעלמות מאחורי החץ «. את Lighthouse מחפשים מאחורי ה-» של שורת הלשוניות העליונה; את Accessibility — בסרגל הצד הימני של Elements (לשוניות Layout · Computed · Styles · ...), ואם הוא צר, מאחורי ה-» שלו.

לשונית	מה עושים בה	מתי בקורס
Elements	העץ שהדפדפן בנה מה-HTML שלך; בחר צומת ו- <code>\$0</code> בקונסול הוא אותו אלמנט	שבוע 1
Console	מקלידים JavaScript ורואים תשובה. <code>('sel' \$)</code> הוא קיצור של DevTools בלבד: מחזיר את כל האלמנטים שמתאימים לסלקטור, ו- <code>length</code> . סופר אותם	שבוע 1
Network	כל בקשה שהעמוד שלח (טען מחדש כשהלשונית פתוחה); רואים 404 וכתובות	שבוע 1 (נתיבים) · שבוע 11 (<code>fetch</code>)
Accessibility	עץ הנגישות: מה קורא מסך מקבל (<code>generic</code> , <code>main</code> , <code>banner</code>). ב-Elements, בסרגל הצד הימני, בלשונית Accessibility (כשהסרגל צר — מאחורי ה-» שלו)	שבועות 1–2
Lighthouse	ביקורת נגישות וביצועים לעמוד (לשונית עליונה; כשהחלון צר — מאחורי ה-» העליון)	שבוע 2
Styles Computed-ו	החלוניות הימניות ב-Elements: הכללים שחלים על האלמנט, ומי ניצח	שבוע 3
Device toolbar	<code>Cmd+Shift+M</code> / <code>Ctrl+Shift+M</code> : מסך של טלפון, ותיבת הרוחב למעלה לגרירה	שבוע 1 (מחזור 3) · מ-4 והלאה
Rendering	שלוש הנקודות האנכיות, אז More tools , אז Rendering : הדמיית <code>prefers-reduced-motion</code> , <code>prefers-color-scheme</code> וליקויי ראייה	שבוע 3
Application	בעץ הצד: Local storage : מה <code>localStorage</code> שמר לעמוד	שבוע 11
Sources	קובץ ה-JS, ונקודות עצירה (breakpoints) לצעוד בקוד שורה-שורה	שבוע 9

בדוק עכשיו (שתי דקות): פתח עמוד כלשהו, פתח את הפאנל, לחץ על Elements, על Console (הקלד `document.title`), ועל Network (טען מחדש). אם ראית את שלושתם — סיימת.

6. גיטהב – החשבון, המאגר, וההרשאה שלי

6.1 פתיחת חשבון

1. גש אל <https://github.com/signup>.
2. השתמש **באותו מייל** שהגדרת ב- `git config user.email`.
3. בחר שם משתמש שתסכים שיופיע לצידך גם בעוד חמש שנים.
4. אשר את המייל.

חשבון אישי חינמי. אין צורך בארגון, אין צורך באימות סטודנט, ואין שום שלב בקורס שדורש תשלום.

6.2 חיבור מאובטח – בחר דרך אחת

גיטהב לא מקבל סיסמאות בפעולות `git push` מאז 2021. יש שתי דרכים תקינות, ומספיקה אחת. אם אין לך העדפה – בחר במסלול SSH; מגדירים אותו פעם אחת והוא לא מטריד יותר.

אפשרות א' – מפתח SSH (מומלץ)

macOS ו-Windows, אותן פקודות:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

לחץ Enter שלוש פעמים (מיקום ברירת מחדל, ואז סיסמה ריקה פעמיים). עכשיו העתק את המפתח הציבורי:

```
# macOS
pbcopy < ~/.ssh/id_ed25519.pub

# Windows (PowerShell)
Get-Content ~/.ssh/id_ed25519.pub | Set-Clipboard
```

בגיטהב: לחץ על התמונה שלך בפינה, ואז `Settings > SSH and GPG keys > New SSH key` (My laptop), הדבק, ושמו.

בדוק:

```
ssh -T git@github.com
```

בפעם הראשונה תישאל: `Are you sure you want to continue connecting (yes/no)?` – ענה `yes`. התשובה הצפויה: `Hi <username>! You've successfully authenticated...`

אפשרות ב' – Personal Access Token

1. בגיטהב: `Settings > Developer settings > Personal access tokens > Tokens (classic)` ואז `Generate new token (classic)`.

2. סמן את ההרשאה `repo` בלבד. תוקף: עד סוף הסמסטר.

3. העתק את הטוקן עכשיו. גיטהב לא יציג אותו שוב לעולם.

4. בפעם הראשונה ש-Git יבקש ממך סיסמה – הדבק את הטוקן במקום הסיסמה.

כדי שלא יישאל שוב:

```
# macOS
git config --global credential.helper osxkeychain

# Windows
git config --global credential.helper manager
```

6.3 יצירת המאגר הפרטי שלך

1. פתח את התבנית של הקורס – הקישור בעמוד הקורס במודל.

2. לחץ `Use this template` ואז `Create a new repository`.

3. שם המאגר: <תעודת הזהות שלך>-2027-frontend – בדיוק כך, באנגלית, עם מקפים.

4. בחר **Private**. לא **Public**.

5. **Create repository**.

למה פרטי: ארבעים מאגרים ציבוריים באותו קורס הם מכונת העתקות. **למה מהתבנית:** התבנית מביאה איתה את כל ההגדרות – `.vscode`, `.gitattributes`, `.editorconfig`, והבדיקות האוטומטיות. מאגר ריק שיצרת לבד לא יכיל אותן, וזה יעלה לך נקודות בכל שבוע.

6.4 הוסף אותי כמשתף פעולה

1. במאגר שלך: **Settings > Collaborators > Add people**.

2. הקלד את שם המשתמש שלי בגיטהאב – הוא רשום על הלוח ובעמוד הקורס.

3. **Add to repository**. ברשימה תופיע שורה עם הסטטוס **Pending invite**.

Pending invite זה בדיוק מה שאמור להופיע, וזה מספיק. בגיטהאב מי שמאשר הזמנה הוא המוזמן, כלומר אני – אתה לא יכול לאשר אותה בעצמך, ולכן אתה גם לא נמדד עליה. אני מאשר את כל ההזמנות באותו ערב.

ההזמנה תקפה כשבוע בלבד. אם עבר שבוע ובמאגר עדיין כתוב **Pending invite** (או שהיא נעלמה מהרשימה), היא פגה: בחר **Settings > Collaborators**, לחץ **Add people** והזמן אותי שוב – שורה חדשה תופיע עם הסטטוס **Pending invite**.

מה כן באחריותך: שאני אופיע ברשימת ה-**Collaborators**. ודא שאתה רואה אותי שם לפני שאתה יוצא מהמעבדה – זה מה שנבדק.

למה זה קריטי: בשבוע 13 אני טוען את הפרויקט שלך למחשבי המבחן לפני שאתה נכנס לחדר. בלי גישה למאגר אני לא יכול לעשות את זה, ואין לזה פתרון ביום המבחן. זו הסיבה שיש על הסעיף הזה נקודות כבר במטלה הראשונה.

6.5 שכפול המאגר, והדחיפה הראשונה

בעמוד המאגר שלך לחץ **Code** והעתק את הכתובת – לשונית **SSH** אם בחרת מפתח, **HTTPS** אם בחרת טוקן.

```
cd ~/Documents
git clone git@github.com:YOUR-USERNAME/frontend-2027-123456789.git
cd frontend-2027-123456789
code .
```

עכשיו הדחיפה הראשונה. פתח את `week-01/README.md`, הוסף שורה כלשהי, ושמור:

```
git status
git add week-01/README.md
git commit -m "docs: first commit from my machine"
git push
```

אם **git push** עבר בלי שגיאה – סיימת את החלק הקשה. רענן את עמוד המאגר בדפדפן וראה את הקומיט שלך שם.

לא עבר? `git remote -v` יגיד לך על איזה פרוטוקול אתה, ונספח א מכסה את כל הכישלונות הנפוצים (א-6 עד א-9).

7. מאגר החומרים הציבורי, ו-`git pull` שבועי

חומרי הקורס – קבצי הפתיחה, הברייפים והמצגות – יושבים במאגר ציבורי נפרד. שכפל אותו **פעם אחת**, לתיקיה **אחרת** מזו של המאגר שלך:

```
cd ~/Documents
git clone https://github.com/<the account named on the course page>/frontend-2027-
materials.git
```

בכל שבוע, לפני השיעור:

```
cd ~/Documents/frontend-2027-materials
git pull
```

זהו. `git pull` מביא את מה שהתווסף. **אל תערוך קבצים בתוך המאגר הזה** – העתק את מה שאתה צריך אל המאגר שלך ועבוד שם. אם ערכת בטעות ו-`git pull` מסרב, ראה תקלה א-9.

8. כלל ההגשה – למד אותו עכשיו, הוא לא ישתנה

הגשה

בכל מועד הגשה אתה מגיש במודל שני דברים: כתובת ה-`repo`, ו-`SHA` של הקומיט.

ה-`SHA` מקפיא את ההגשה. מה שנמצא בקומיט הזה – זה מה שנבדק. דחיפה אחרי המועד לא משנה את הציון, ואין על כך ויכוח.

איך משיגים `SHA`

מהטרמינל:

```
git log -1 --format=%H
```

התוצאה היא שורה אחת של 40 תווים, למשל:

```
9f4c1a7e0b2d5836a1c4f7e9d2b8a6c3e5f10947
```

מגיטהב: בעמוד המאגר לחץ על `commits`, ולצד כל קומיט יש כפתור העתקה קטן שמעתיק את ה-`SHA` המלא.

חזרה יבשה – עשה את זה היום

1. ודא שדחפת (`git push`).

2. הרץ `git log -1 --format=%H` והעתק את הפלט.

3. פתח את שאלת ההגשה של שבוע 1 במודל.

4. הדבק את כתובת ה-`repo` ואת ה-`SHA`, והגש.

Claude Code 9.

לא נחוץ עד שבוע 8 — ודורש מני בתשלום

Claude Code דורש מני בתשלום (Claude Pro ומעלה, או מפתח API). אין בקורס שום דבר שדורש אותו: המסלול החינמי — הצ'אט בדפדפן — **נבדק ומנוקד בדיוק באותה מידה**, בכל מטלה, עד סוף הסמסטר. מי שיש לו מני מתקין עכשיו, כי אני כאן לידו ולא בשבוע 8 תחת לחץ זמן; מי שאין לו — מדלג על הסעיף הזה בלב שלם. **אסור להשתמש בו למטלות בשבועות 1-7**. בשבועות 1-5 AI הוא מורה פרטי בלבד: מותר לבקש הסבר או ביקורת על קוד שכתבת, ואתה מקליד כל תו בעצמך; בשבועות 6-7 הוא עוזר שמציע גרסאות ואתה בוחר ומנמק. שכבת "שותף למימוש" — שבה Claude Code בכלל רלוונטי — מתחילה בשבוע 8.

התקנה (שני המסלולים)

צריך שיהיה לך Node מפרק 4. פתח את הטרמינל **המשולב של VS Code** (`Ctrl+`` או `Cmd+``):

```
npm install -g @anthropic-ai/claude-code
```

ואז, מתוך תיקיית הפרויקט:

```
claude
```

בהרצה הראשונה תתבקש להתחבר בדפדפן. אשר, וחזור לטרמינל.

אם קיבלת `npm ERR! code EACCES` במק — ראה תקלה א-13. אם `claude` לא נמצא אחרי ההתקנה — סגור את הטרמינל ופתח חדש (א-14).

אין לך מני? אין בעיה

אין לך מני? הכול עובד גם בצ'אט החינמי, ואין בכך שום חיסרון בקורס.

אף שלב בקורס לא חוסם אותך בלי מני, והציון זהה. בכל מקום שבו כתוב "הרץ את Claude Code" יש מסלול חלופי מלא: פתח את הצ'אט החינמי בדפדפן, הדבק לתוכו את המפרט שכתבת, והעתק את הקבצים שחזרו לתוך העורך ידנית. **שלב הביקורת נשאר זהה** — אתה קורא כל שורה ומחליט אם היא נכונה, וזו בדיוק המיומנות שנבחנת.

נספח א — התקלות שבאמת קורות

חפש את הודעת השגיאה שלך בטבלה הזו. ההודעות כתובות כאן בדיוק כפי שהן מופיעות על המסך.

macOS • 1-א

```
zsh: command not found: node
```

הטרמינל לא יודע איפה Node הותקן. **תיקון:** סגור את כל חלונות הטרמינל ופתח חדש. אם זה לא עזר, ההתקנה כנראה נכשלה — התקן שוב את חבילת ה-LTS מ- `nodejs.org` ושים לב שההתקנה מסתיימת בהצלחה. אפשר לוודא איפה Node נמצא עם `ls /usr/local/bin/node`.

א-2 • Windows

```
'node' is not recognized as an internal or external command,  
operable program or batch file.
```

Node הותקן, אבל הטרמינל שלך עלה **לפני** ההתקנה ולכן לא ראה את השינוי ב-PATH. **תיקון:** סגור לגמרי את VS Code (לא רק את הלשונית) ופתח מחדש. אם עדיין לא — התקן מחדש וודא שהתיבה `Add to PATH` מסומנת.

א-3 • שני המסלולים

```
zsh: command not found: git  
'git' is not recognized as an internal or external command
```

Git לא מותקן, או לא ב-PATH. **תיקון:** חזור לפרק 5 והתקן. בחלונות — סגור ופתח מחדש את VS Code אחרי ההתקנה.

א-4 • שני המסלולים

```
fatal: not a git repository (or any of the parent directories): .git
```

אתה מריץ פקודת Git בתיקייה שאינה מאגר. **תיקון:** בדוק איפה אתה עם `pwd` (מק) או `cd` (חלונות). כמעט תמיד פתחת ב-VS Code תיקייה מעל או מתחת למאגר. פתח את התיקייה שיש בה `.git` — זו התיקייה שקיבלת מ- `git clone`.

א-5 • שני המסלולים

```
Author identity unknown  
  
*** Please tell me who you are.  
  
Run  
  
git config --global user.email "you@example.com"  
git config --global user.name "Your Name"
```

לא הגדרת שם ומייל. **תיקון:** הרץ בדיוק את שתי הפקודות שהודעה מציעה, עם הפרטים שלך. זה נדרש פעם אחת במחשב.

א-6 • שני המסלולים

```
remote: Support for password authentication was removed on August 13, 2021.
fatal: Authentication failed for 'https://github.com/...'
```

אתה עובד ב-HTTPS ומנסה סיסמה. גיטהב לא מקבל סיסמאות. **תיקון:** צור Personal Access Token (פרק 6.2, אפשרות ב') והדבק אותו במקום הסיסמה. או עבור ל-SSH:

```
git remote set-url origin git@github.com:YOUR-USERNAME/YOUR-REPO.git
```

א-7 • שני המסלולים

```
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.
```

אתה עובד ב-SSH, אבל המפתח לא מוכר לגיטהב. **תיקון:** ודא שהעלית את המפתח הציבורי (`id_ed25519.pub` , זה שמסתיים ב- `.pub`) למסך SSH and GPG keys. בדוק עם `ssh -T git@github.com`.

א-8 • שני המסלולים

```
remote: Repository not found.
fatal: repository 'https://github.com/...' not found
```

שלוש סיבות אפשריות, לפי הסדר:

1. טעות כתיב בכתובת. השווה מול הכפתור `Code` בעמוד המאגר.
2. אתה מחובר לחשבון גיטהב אחר משלך.
3. המאגר פרטי ואין לך גישה אליו — קורה כשמנסים לשכפל מאגר של מישהו אחר.

א-9 • שני המסלולים

```
error: Your local changes to the following files would be overwritten by merge:
      README.md
Please commit your changes or stash them before you merge.
```

יש לך שינויים שלא נשמרו בקומיט, ו- `git pull` לא רוצה לדרוס אותם. **תיקון:** אם השינויים חשובים — `git add` ואז `git commit -m "..."` ואז `git pull` ואם הם לא — `git restore` ואז `git pull`. פירוט מלא ב- `git-panic-he.md`.

א-10 • Windows

```
warning: LF will be replaced by CRLF in index.html.
The file will have its original line endings in your working directory
```

זו לא שגיאה. זו הודעה ש-Git עושה בדיוק את מה שהוא אמור: מנרמל את סופי השורה כדי שההגשה שלך תיראה זהה גם במק. אפשר להתעלם.

א-11 • שני המסלולים

```
Cannot GET /
```

Live Server רץ, אבל אין `index.html` בשורש התיקיה שפתחת. **תיקון:** פתחת תיקיה גבוהה מדי. `File > Open Folder...`, ובחר את התיקיה שבה יושב `index.html` עצמו.

א-12 • שני המסלולים

האפשרות **Open with Live Server** לא מופיעה בתפריט הימני.

התוסף לא מותקן, או שפתחת קובץ בודד ולא תיקיה. **תיקון:** `Cmd/Ctrl+Shift+X`, חפש `@recommended`, והתקן. ואז `File > Open Folder...`. אם הפורט 5500 תפוס על ידי תוכנה אחרת, VS Code יודיע ויעבור ל-5501 — סגור את התוכנה האחרת וחזור ל-5500, כדי שהנתונים של שבוע 11 לא יתבללו.

א-13 • macOS

```
npm ERR! code EACCES
npm ERR! syscall mkdir
npm ERR! Error: EACCES: permission denied
```

התקנה גלובלית מנסה לכתוב לתיקיה שאין לך אליה הרשאה. **אל תריץ** `sudo npm install` — זה יוצר בעיות הרשאות גרועות יותר בהמשך. **תיקון:** הגדר תיקיית התקנה גלובלית משלך:

```
mkdir -p ~/.npm-global
npm config set prefix ~/.npm-global
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.zshrc
source ~/.zshrc
```

ואז התקן שוב.

א-14 • שני המסלולים

```
zsh: command not found: claude
'claude' is not recognized as an internal or external command
```

ההתקנה הצליחה אבל הטרמינל לא רואה אותה עדיין. **תיקון:** סגור את הטרמינל המשולב ופתח חדש. בחלונות — סגור את VS Code כולו ופתח מחדש.

נספח ב – רשימת בדיקה סופית

זה הדבר הראשון שנבדק בסמסטר. הרץ את חמש הפקודות, סמן את חמש התיבות, וצלם מסך אחד.

הפקודות

פתח את הטרמינל המשולב של VS Code, בתוך תיקיית המאגר שלך, והרץ:

```
node -v
npm -v
git --version
git config --global user.name
git log -1 --format=%H
```

חמש שורות פלט, ואף אחת מהן אינה שגיאה.

הבדיקות

חמש בדיקות. הראשונות ארבע הן על המחשב שלך, והן בדיוק מה שצריך להיראות בצילום המסך שבסעיף הבא. החמישית והשישית הן בגיטהאב, בדפדפן, ואותן אני בודק ישירות במאגר שלך – הן לא צריכות להיות בצילום.

- ☐ . VS Code פתוח על תיקיית המאגר שלך, ובסייר הקבצים רואים את `week-01/`.
- ☐ . הטרמינל המשולב מציג את הפלט של חמש הפקודות שלמעלה, בלי שגיאות.
- ☐ . העמוד רץ בדפדפן, ובשורת הכתובת כתוב `http://127.0.0.1:5500` – לא `file://`.
- ☐ . לחצת על סמל התוספים ורואים שמונה תוספים מותקנים.
- ☐ . בעמוד המאגר בגיטהאב רואים את הקומיט שלך, והמאגר מסומן **Private**.
- ☐ . במסך **Collaborators** רואים אותי.

צילום המסך שמגישים

ארבעה דברים צריכים להיראות. מותר עד שני צילומי מסך.

1. VS Code פתוח על תיקיית המאגר שלך – בסייר הקבצים רואים את `week-01/`.
2. הטרמינל המשולב, עם הפלט של חמש הפקודות מנספח ב – חמש שורות, אף אחת מהן לא שגיאה.
3. הדפדפן עם העמוד שלך, והכתובת `http://127.0.0.1:5500` גלויה בשורת הכתובת.
4. פאנל התוספים, עם שמונת התוספים מותקנים.

למה עד שניים: על מסך של 13 אינץ' אי אפשר להצטלב את כל הארבעה בחלון אחד קריא. חלק נפוץ: $1+2+3$ בצילום אחד, ופאנל התוספים בשני. אין עדיפות לחלוקה אחת על פני אחרת – רק שהארבעה ייראו, ושאפשר לקרוא את הטקסט.

צרך להגשה של מטלת כיתה 1. זה שווה 6 נקודות – לא כדי להציק, אלא כדי שאף אחד לא יגיע לשבוע 7 עם סביבה שבורה.

נתקעת ב-Git ואינך מוצא את המצב שלך בנספח א? `git-panic-he.md` מכסה את חמשת המצבים שבאמת קורים – שינויים שחוסמים `pull`, קונפליקט מיזוג, קומיט שגוי, `HEAD` מנותק, ודחיפה לענף הלא נכון – עם הפקודות המדויקות לצאת מכל אחד.